Campus Café — Azure serverless order tracking

Simplified architecture v1 · George Brown College · Work Integrated Project · October 2026

3 functions | 1 storage account | 1 email service | 4 statuses · 3 emails | est. under $1 / month

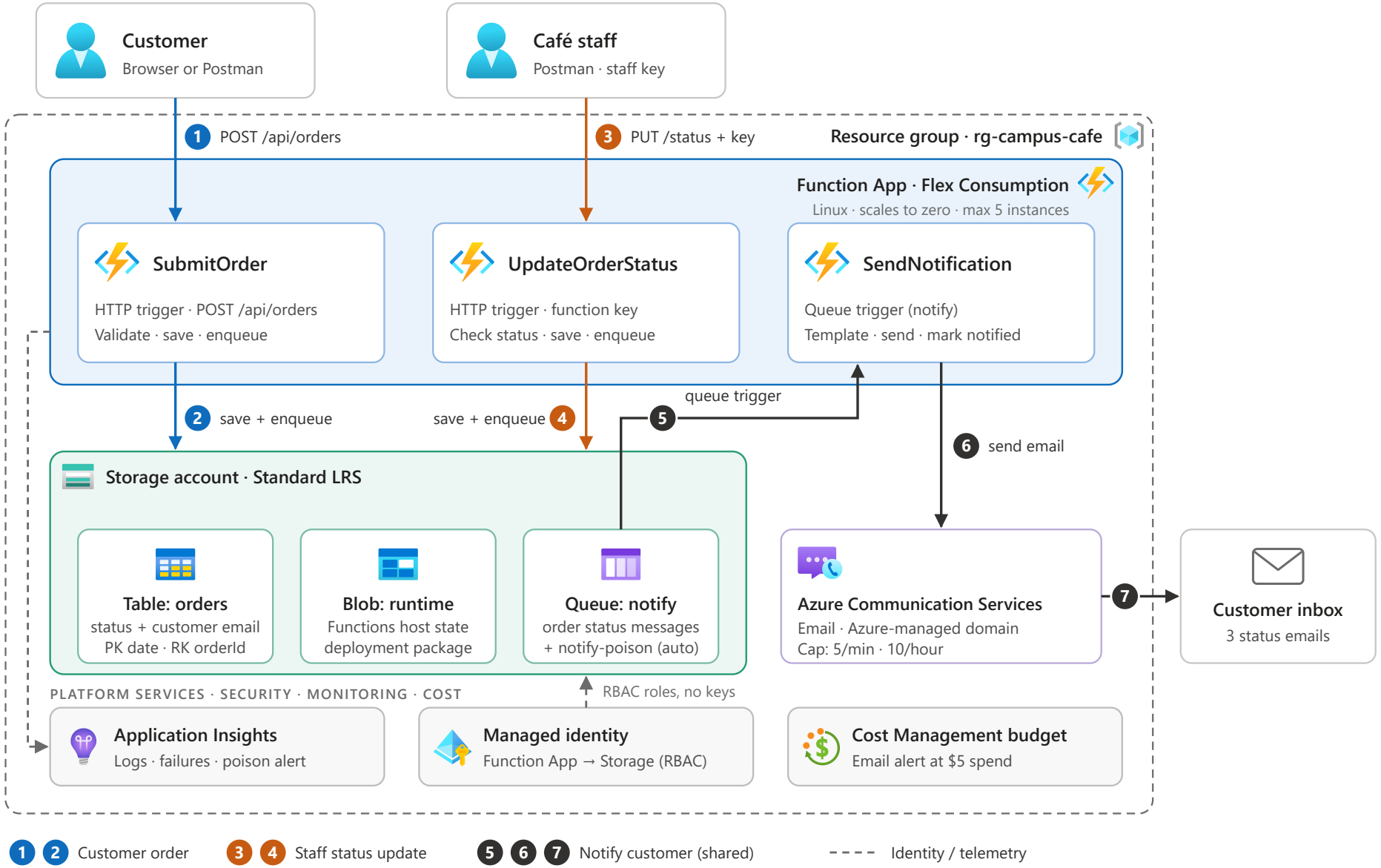


Figure 1. Customer order = steps 1–2 (blue). Staff status update = steps 3–4 (orange). Notification = steps 5–7 (shared by both paths). Dashed lines = identity and telemetry. Azure icons: Microsoft Azure Public Service Icons V24, see References on page 3.

How it works

What each piece does, why the design is this small, and what it costs

What it does

A customer places a coffee order and gets an email as it moves through four statuses: **Accepted → Preparing → Ready for pickup → Collected**. Staff move the status forward. The customer receives three emails. **Collected** closes the order with no email.

Steps (numbers match Figure 1)

1. Customer sends `POST /api/orders` over HTTPS. The endpoint is public, so it validates input and limits size.
2. `SubmitOrder` saves the order as `Accepted` in Table `orders`, puts `{orderId, status}` on Queue `notify`, returns the `orderId`.
3. Staff send `PUT /api/orders/{id}/status` with a function key.
4. `UpdateOrderStatus` allows only the next valid status, saves it, and enqueues a message. `Collected` is saved with no message.
5. The queue message triggers `SendNotification`. A failed message retries; after 5 failures it moves to `notify-poison`.
6. `SendNotification` picks the email text for the status, sends it through Azure Communication Services Email, then marks that status as notified so duplicates are skipped.
7. Customer receives Accepted, Preparing and Ready emails.

Why this shape

- **Serverless:** pay per execution, scales to zero, no servers to patch.
- **Event-driven:** a queue message triggers the email function, so HTTP calls return fast and email trouble never blocks an order.
- **Queue as shock absorber:** the email service has hard send limits. Messages wait and retry instead of failing the request.
- **Minimal on purpose:** the earlier V24 design (`ProcessOrder` + Event Grid) is deferred until a second consumer exists.

Services and cost (estimate)

Service	Role	Cost
Azure Functions, Flex Consumption	3 functions, scale to zero	Free grant: 250,000 executions + 100,000 GB-s per month
Storage account, Standard LRS	Table, Queue, Blob runtime	Cents per month
Azure Communication Services Email	Customer emails	$0.00025 per email
Application Insights	Logs, failures, poison-queue alert	First 5 GB per month free
Managed identity	Function App to storage via RBAC	Free
Cost Management budget	Alert before overspend	Free

Total estimate: under $1 per month, covered by Azure for Students ($100 credit, no card). Figures from Microsoft pricing pages, Oct 2026. Confirm in the Azure Pricing Calculator.

Well-Architected check

Pillar	In v1	Add later
Reliability	Queue retry, poison queue, idempotent email, stateless functions	ZRS/GRS, Event Grid dead-letter
Security	HTTPS only, function key for staff, managed identity to storage, no personal data in logs	Entra ID sign-in, Key Vault, VNet + private endpoints
Cost optimization	Scale to zero, free grants, LRS, instance cap, budget alert	—
Operational excellence	Application Insights, poison-queue alert, Git repo + README runbook	Bicep or Terraform, GitHub Actions deploy
Performance efficiency	Email off the request path, per-function scaling	Always-ready instance, caching

Trade-offs, watch-outs and next steps

Honest limits of the simple design, verified against Microsoft documentation on 14 Oct 2026

Good

- Few parts: 3 functions, 1 storage account, 1 email service. Easy to explain and demo.
- Still event-driven and decoupled: queue trigger, retries, poison queue.
- Near-zero cost, scales to zero.
- Same features as V24: 4 statuses, 3 emails.
- Clear upgrade path: each later step is one add-on.

Bad

- `SubmitOrder` is public (spam risk). A function key is a shared secret, not a user identity.
- Email is capped, may land in spam, and the service is retiring.
- Table Storage has no auto-number and weak queries. Staff cannot list open orders yet.
- Email only. The customer has no order-status page.
- Save and enqueue are not atomic. The notified flag makes retries safe.
- Cold start of a few seconds after idle. Single region, LRS only.

Later phases

Event Grid when a second consumer exists · `GET /orders` staff board · Entra ID sign-in · Key Vault · VNet + private endpoints · Bicep + GitHub Actions · simple web page on Static Web Apps (Free).

Watch-outs

- **ACS Email retires 30 Sep 2028** (announced Sep 2026). Microsoft limits onboarding to retiring ACS services from 23 Oct 2026 and advises migrating rather than building new. Keep email behind `SendNotification` and an `EMAIL_MODE` setting so a swap touches one function. Alternatives: Marketplace partners (Infobip, Telesign). M365 High Volume Email is internal-send only.
- **Azure-managed domain cap: 5 emails per minute, 10 per hour per subscription, no increase.** Three emails per order is about 3 orders per hour. A verified custom domain allows 30 per minute, 100 per hour.
- **Flex Consumption is Linux only.** Student subscriptions get a 15-core default regional quota, plenty for 3 functions.

Build order (each step demo-able)

1. Resource group, storage account, Function App (Flex), Application Insights. Hello-world HTTP function.
2. `SubmitOrder` writes to Table. Test in Postman.
3. Queue + `SendNotification` with `EMAIL_MODE=log`.
4. `UpdateOrderStatus` with status rules.
5. Switch to `EMAIL_MODE=acs`.
6. Harden: managed identity, function key, HTTPS only, $5 budget, log daily cap.

Develop locally first with Azurite (free storage emulator) and log mode. Cost stays $0 until deploy.

References and credits

- **Icons:** Microsoft Azure Public Service Icons V24, file `Azure_Public_Service_Icons_V24.zip` (Azure architecture icons, © Microsoft). Used unmodified (no crop, flip, rotate or distortion) with the product name beside each icon, for architecture documentation, as the pack's Terms of Use permit. Icons used: Users, Resource Groups, Function Apps, Storage Accounts, Table, Blob Block, Storage Queue, Azure Communication Services, Application Insights, Managed Identities, Cost Budgets.
- **Microsoft documentation:** Flex Consumption plan · Functions pricing · ACS service limits · ACS Email pricing · ACS retirement guide · Azure for Students